Assignment 1

1. List 3 common hacking methodologies (such as phishing, brute force, SQL injection) and for each, write one real-world example of how it could be used to compromise an account on an app like Instagram or Paytm.

Ans-

1. Phishing

Meaning: An attacker tricks a user into entering their login details on a fake website.

Example: A user receives a fake Instagram security alert. They open a fraudulent login page and enter their password, exposing it to the attacker.

Prevention: Check the URL and use multifactor authentication (MFA).

2. Brute-force attack

Meaning: A program repeatedly tries different password or PIN combinations.

Example: An attacker tries to guess a weak four-digit PIN on a poorly protected test login system. A real payment app should use attempt limits and additional security controls.

Prevention: Rate limiting, temporary lockouts, strong authentication, and MFA.

3. SQL injection (SQLi)

Meaning: An attacker supplies specially crafted input to exploit unsafe database queries.

Example: A poorly coded food-delivery website builds a database query directly from user input. The attacker may bypass a login check or access data they should not be able to see.

Prevention: Parameterized queries, input validation, and least-privilege database accounts.

**2. Simulate a simple brute force attack by writing a Python script that tries all 4-digit PIN combinations (0000-9999) to guess a preset password variable. Print the number of attempts taken to find the correct PIN.

Constraint: Do not use external libraries for this task.**

Ans-

```
# Preset target PIN (Change this to test different combinations)
PRESET_PIN = "4732"
# Simulation variables
attempts = 0
```

```
found_pin = None
# Brute force loop from 0000 to 9999
for i in range(10000):
    attempts += 1
    # Format the number as a 4-digit string with leading zeros (e.g., 5 -> "0005")
    guessed_pin = f"{i:04d}"
    # Check if the guess matches the preset PIN
    if guessed_pin == PRESET_PIN:
        found_pin = guessed_pin
        break
# Output results
print(f"Success! Correct PIN found: {found_pin}")
print(f"Total attempts taken: {attempts}")
```

3.Research and describe the steps involved in a phishing attack targeting users of a popular food delivery app like Zomato or Swiggy. Include at least 4 steps and explain what the attacker and victim do at each stage.

Ans-

Phase 1: Bait Creation & Infrastructure Setup

- Attacker's Action: Register a typosquatted domain name (e.g., swiggy-refunds.com or zomato-discounts.net) and deploy a cloned landing page mimicking the official app's portal.
- Victim's Action: Passive at this stage; they have no awareness that a malicious domain mimicking their trusted app has been created.

Phase 2: Distribution & Luring

- Attacker's Action: Send mass SMS broadcasts (Smishing) or emails claiming the target has won a "Free ₹500 voucher" or needs to "Claim a refund for a failed transaction" with a link to the fake domain.
- Victim's Action: Receive the notification, feel enticed by the financial incentive or urgent refund claim, and click the malicious link.

Phase 3: Credential Harvesting & Exploitation

- Attacker's Action: Host a malicious form on the landing page that captures inputs in real time. Once credentials or card details are typed, a script routes them to the attacker's server.
- Victim's Action: Enter their official login phone number, password, or bank card details on the fake website believing they are authenticating with the app.

Phase 4: Session Hijacking & Exfiltration

- Attacker's Action: Trigger an automated script to inject the harvested credentials into the legitimate app. If an OTP is generated, the fake site prompts the victim for it, allowing the attacker to drain wallet balances or make unauthorized purchases.
- Victim's Action: Enter the OTP into the malicious prompt, losing access to their account or funds, and are often redirected to the actual application to mask the fraud.

**4.Given the following fake login page HTML code, identify and explain two ways a hacker could use this page for credential harvesting.

Hint: Think about what happens to the data entered by the user and how it could be sent to an attacker.**

Ans-<!DOCTYPE html>

<html>

<head>

<title>Secure Login Portal</title>

<style>

body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: center; height: 100vh; background-color: #f4f4f4; }

.login-box { background: white; padding: 30px; border-radius: 8px; box-shadow: 0px 0px 10px rgba(0,0,0,0.1); }

input { display: block; width: 250px; margin: 10px 0; padding: 10px; }

button { width: 100%; padding: 10px; background: #007bff; color: white; border: none; border-radius: 4px; cursor: pointer; }

</style>

</head>

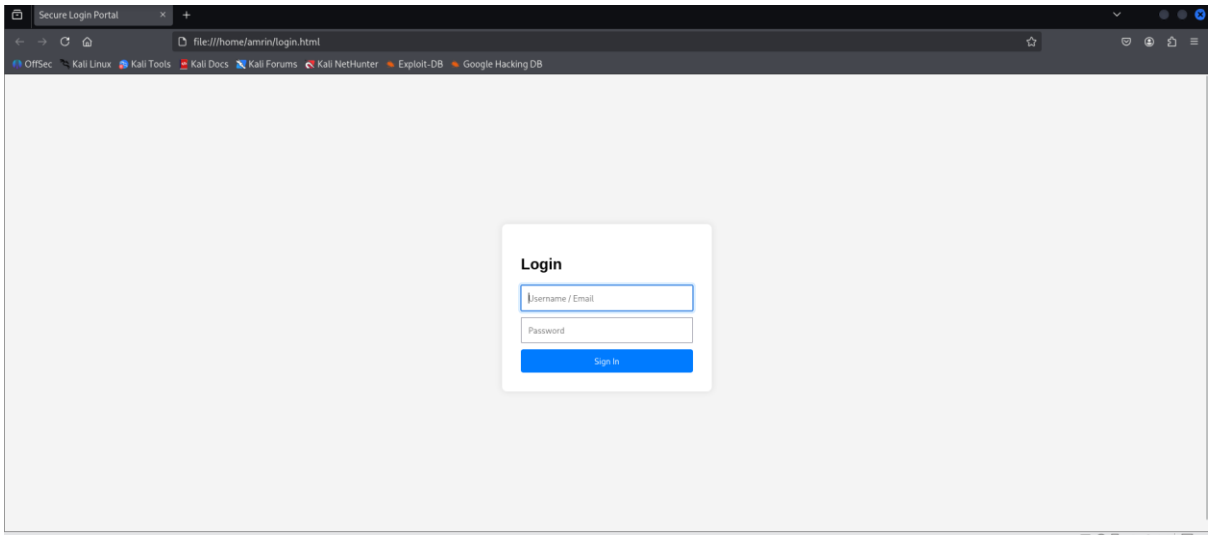
<body>

<h2>Login</h2>

<!-- THE FORM TAG EXPLAINED IN METHOD 1 -->

```
        <form id="loginForm" action="http://127.0.0" method="POST">
            <input type="text" id="username" name="username" placeholder="Username / Email" required>
            <input type="password" id="password" name="password" placeholder="Password" required>
            <button type="submit">Sign In</button>
        </form>
    </div>

    <!-- THE JAVASCRIPT INJECTION EXPLAINED IN METHOD 2 -->
    <script>
        const passwordField = document.getElementById('password');
        passwordField.addEventListener('input', (e) => {
            // Simulating a real-time keylogger sending data out silently
            console.log("Attacker Intercepted Keystroke: " + e.target.value);
        });
    </script>
</body>
</html>
```



5.Use ChatGPT or another AI tool to generate a list of 5 recent real-world hacking incidents involving Indian apps or websites. For each incident, write one line about the hacking methodology used.

Ans-

- boAt Lifestyle Data Breach: Attackers dumped sensitive customer records on the dark web after bypassing API endpoints or database access controls to extract personally identifiable information (PII).
- Taj Hotels (IHCL) Incident: Threat actors used compromised administrative access tokens to pull customer data structures directly from the group's central reservation systems.
- Freecharge (Axis Bank) Unauthorized Access: Attackers utilized credential stuffing methodologies, validating leaked password lists against exposed account verification endpoints to compromise user wallets.
- RailYatri Data Leak: Malicious actors exploited an unsecured, publicly exposed Elasticsearch server instance to download passenger travel details and PII without requiring authentication.
- CoWIN Portal Data Exposure via Telegram Bot: Attackers set up automated scraping scripts that queried internal verification API keys, allowing them to pull sensitive identity data programmatically via user phone numbers.